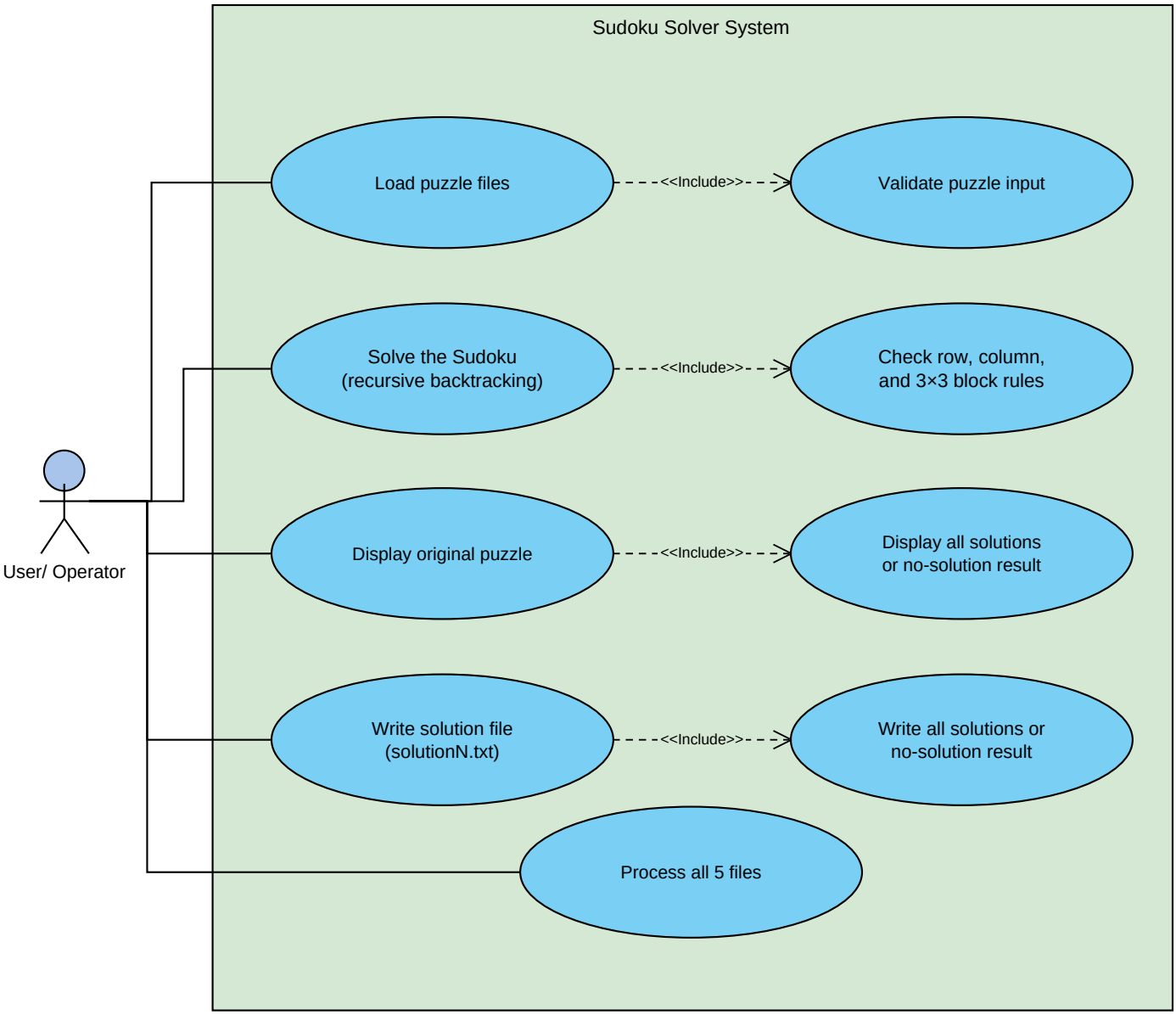

Assignment 5 revision: new use case "Write solution file (solutionN.txt)" and the use case it includes.
Diagram modified from the Assignment 4 diagram received; created with GenAI assistance (Claude).

Sudoku Solver – Revised Use Case Model (Assignment 5)

Use case added to the Assignment 4 diagram: **Write solution file (solutionN.txt)**

Diagram and write-up created with GenAI assistance (Claude).

A: Use Case

Use case name: Write Solution File (solutionN.txt)

Primary actor: User/ Operator

Secondary actors: None on the diagram. The solution files (solution1.txt through solution5.txt) are created by the program.

The Assignment 5 requirements add one new user-visible goal: given Sudoku puzzle files, the program must save the solutions for each puzzle to a file named solutionN.txt, not only print them. Loading, validating, solving, and displaying were already modeled in Assignment 4, so only one use case is new. Write Solution File is that use case, and the step that decides what goes into the file is modeled as the supporting use case it includes, Write all solutions or no-solution result.

B: Brief Description

After the system finds the solutions to a puzzle, it saves them to solutionN.txt, where N matches the puzzle file (puzzle3.txt gives solution3.txt). If the puzzle has solutions, the file holds every solution board in the same format as the puzzle files, followed by one line, “Solutions found: n.” If the puzzle has no solution, the file holds the single line “No solution found.” The results are still printed to the console as in Assignment 4. If the output file cannot be created, the system reports the error and moves on to the next puzzle.

C: Step-by-Step Flow

Main flow

1 The User runs the program.

2 The system takes the next puzzle file (puzzle1.txt through puzzle5.txt) and loads and validates it (*Load puzzle files*, *Validate puzzle input*).

3 The system prints a separator line, the puzzle file name, and the original puzzle to the console.

4 The system creates the output file solutionN.txt, where N is the number of the puzzle file.

5 The system checks that the given digits do not already break a row, column, or 3×3 block rule.

6 The system searches recursively with backtracking. Each time it fills the last empty cell, it counts a solution, prints the board to the console, and writes the board to solutionN.txt as nine rows of nine space-separated digits followed by a blank line.

7 When the search is exhausted, the system writes “Solutions found: n” as the last line of solutionN.txt, where n is the number of solutions, and prints the same line to the console.

8 The system repeats steps 2 to 7 for each remaining puzzle file (*Process all 5 files*).

Alternative flows

2a. The puzzle file cannot be read or is not a valid puzzle. The system prints an error naming the file and skips it. No solutionN.txt is created for that puzzle. The flow continues at step 8.

4a. The output file cannot be created. The system prints “Error: Could not create solutionN.txt.” and skips solving that puzzle. The flow continues at step 8.

6a. The puzzle has no solution (including when the given digits conflict). Nothing is written during the search. The flow continues at step 7a.

7a. The system writes the single line “No solution found.” to solutionN.txt and prints the same message to the console. The flow continues at step 8.

6b. The puzzle has more than one solution. Every solution is written, each followed by a blank line, before the “Solutions found: n” line.

D: Detailed Description

Use case name: Write Solution File (solutionN.txt)

Primary actor: User/ Operator

Secondary actors: None on the diagram; the solution files are written by the program

Goal: Save every solution found for each Sudoku puzzle file, or the fact that there is none, to a file named solutionN.txt, so the results can be checked after the program ends.

Trigger: The User runs the compiled program. The use case runs once for each of the five puzzle files, after the puzzle has been read and displayed.

Included use cases

Write all solutions or no-solution result: writes each solution board in the puzzle file format (nine rows of nine space-separated digits, with a blank line after each solution), followed by the line “Solutions found: n.” If the search finds no solution, it writes the single line “No solution found.” instead.

Extending use cases

None are shown on the diagram. The conditions that change the flow (the puzzle file cannot be read, the output file cannot be created, the puzzle has no solution, the puzzle has several solutions) are described as alternative flows in Section C.

Success end condition: For every puzzle file that was read, a file solutionN.txt exists. It holds either all solutions followed by “Solutions found: n” or the single line “No solution found.” The console shows the same results.

Failure end condition: A puzzle file that cannot be read is reported with an error message and skipped, and no solutionN.txt is created for it. If solutionN.txt cannot be created, the program prints “Error: Could not create solutionN.txt.” and skips solving that puzzle. The remaining puzzles are still processed.

Main success scenario: Steps 1 to 8 in Section C.

Special requirements and rules

The file name is solution + the puzzle number + .txt (puzzle3.txt gives solution3.txt).

Solutions use the same format as the puzzle files. A completed solution has no blanks, so no “_” appears in it.

The last line is exactly “Solutions found: n” or exactly “No solution found.” A puzzle whose given digits conflict has no solution, so its file holds “No solution found.”

Puzzles with several solutions show every solution, one after another. There is no limit on how many are written.

The worst-case running time of the search is $O(9^k)$ for k empty cells. Writing the file adds one extra print of each solution.